Strands Decider

模型原理与支付业务演示

从模型选择，到实际执行，再到可观察的反馈闭环

模型

Pointer Head / 动态候选

Decider 、 GPT 与 BERT 的区别

项目

私有 S3 + CloudFront OAC

ECS + SageMaker + Bedrock

新版演示

逐轮输入输出 / 公共 GPU 窗口

真实固定对比 + 自主闭环

基于原版新建 · 原始 PPT 保留 · 前端 crypto10 · 真实截图与观测均标注来源

把业务流程里的“下一步选择”独立出来

面向快速实验与本地开发的小型开源决策模型；支持 CPU / GPU ，输出选项、是非或评分。

高频、重复

这条消息属于哪个队列?
下一步该调用哪个工具?
证据是否足够继续?

结构化输出

在给定候选项中选一项
返回分布与 confidence
支持多个问题共享上下文

适合嵌入流程

将输出映射为应用动作
记录低置信与参考差异
保留升级到大模型或人工的入口

自然语言理解与路径选择 → 模型 | 金额、权限与执行前置条件 → 应用代码

原理：从文本生成头，切换到候选项评分头

官方 v19： Qwen3.5-2B 主体 + rank-16 LoRA + 约百万参数的 Pointer Head 。

移除 LM head

业务输入

state / question
options / criteria

→

预训练模型主体

Qwen3.5-2B-Base
rank-16 LoRA 微调

→

Pointer Head

候选位置 ↔ <answer>
对每个候选项评分

→

结构化输出

选项分布与选择
是非概率 / 有序评分

一次前向计算

共享状态，批量回答多个问题；减少逐 token 文本生成的开销。

能力边界：不擅长开放式长文本生成与复杂多步推理；任务变长、变复杂时需要重新评估。

一个 choice 判断，怎样从输入变成输出?

以下是官方代码可核对的计算步骤；它不会输出逐条推导的自然语言思维链。

01 组织输入
状态 + 问题
候选名称与条件

→

02 模型编码
读取上下文
得到各位置表示

→

03 候选评分
answer 与选项
通过 Pointer Head

→

04 概率归一化
温度校准 T
softmax 得到分布

→

05 选择与统计
取最大概率候选
计算 confidence

候选分布决定 choice

候选得分 → p = softmax(logits / T)
choice = 概率最大的候选项
不需要生成、再解析一段答案文本

confidence 来自输出分布

choice： 归一化最大候选概率
$(K \times p_max - 1) / (K - 1)$， $K > 1$
表示相对集中程度；仍需业务域校准

候选描述是模型读取的文本条件；它们不会自动变成必须满足的程序约束。

三种问题类型，表达三类业务判断

同一份业务状态可以承载多个问题。候选名称与条件由应用提供。

共享 state： “ Help! My payouts have been failing for 3 days!”

choice · 多选一

该由哪个团队处理?

billing

billing 0.846 / retail 0.090
sales 0.064
confidence = 0.769

noul · 是 / 否

是否表达了紧迫性?

P(true) = 0.829

业务可将概率映射为升级条件
阈值需要独立验证
不是一段生成式解释

score · 有序等级

客户有多沮丧?

score = 1.10

calm / frustrated / depressed
等级分布映射为评分
用于排序与优先级

本页数字来自官方示例；支付案例的实测结果在后续页面单独展示。

Decider 与普通大模型：能力边界不同

本项目让两者完成同一个有限候选决策任务，以便比较行为与耗时。

维度	Strands Decider	普通生成式大模型 / GPT
输出空间	从给定选项选择 / 是非 / 评分	可生成文本；本演示约束为候选 ID
核心计算	模型主体 + Pointer Head 评分	生成式推理，再解析决策 JSON
结果观测	choice 、分布、 confidence	choice 、输入 / 输出 tokens
强项	高频、候选明确的语义判断	复杂解释、规划、写作与开放式任务
运行方式	本项目自行部署 SageMaker GPU	本项目调用 Bedrock 托管模型
共同要求	输入质量、评估、执行约束	输入质量、评估、执行约束

本次没有比较自由对话、开放式 Agent 规划或客服长文本写作质量。

与 BERT 分类器的关键区别：标签含义放在哪里？

这里比较常见的“ BERT 编码器 + 固定标签分类头”； BERT 本身属于深度学习。

常见 BERT 监督分类

输入文本 / 文本对

↓

BERT → [CLS] 池化表示

↓

固定 K 维分类头： $z = W h + b$

↓

类别 1 / 类别 2 / … / 类别 K

Decider 动态候选评分

状态 + 问题 + 本次候选及条件

↓

Qwen 主体 → answer / 候选表示

↓

共享 Pointer Head ： 逐候选评分

↓

本次候选 A / B / … 的概率与选择

共同点：神经网络打分、概率输出与数据驱动训练。 Decider 不是确定性规则引擎。

BERT 与 Decider：任务适配方式不同，需按业务实测

下表比较典型方案，不代表 BERT 只能做固定分类，也不构成性能排名。

维度	常见 BERT 固定标签分类器	Strands Decider v19
主体与规模	双向编码器； BERT-base 约 110M	Qwen3.5-2B 主体 + LoRA + Pointer Head
任务定义	常见方案：标签含义由映射与训练确定	问题、候选名称与条件随本次请求输入
分类头	固定 K 类输出，分类头维度依赖 K	共享投影评分；头的参数量不随 K 增长
新增候选	通常需要扩展分类头、补标注与再训练	可改请求中的候选；仍需验证新任务效果
领域适配	常用领域标注进行监督微调	已有通用决策训练；领域微调仍可能必要
概率输出	softmax 分布；可另外校准	温度校准分布； choice confidence 为统计量
适用重点	标签稳定、领域数据充分、追求轻量部署	候选与任务多变、希望以自然语言配置

BERT 也能用文本对 / 蕴含方式支持动态候选；本项目未实测 BERT 的速度、质量或成本。

适用场景：候选清晰、调用频繁、结果可检查

官方列出的方向包括路由、工具选择、评估、守卫、记忆与上下文管理、策略分类。

模型 / 工具路由

选择模型档位、工具或业务队列

工具调用前检查

参数是否有依据？是否需要澄清？

客服分流与升级

识别诉求，选择继续跟进或转人工

策略分类与审核

将自然语言归入既定策略类别

RAG / 内容评估

判断依据相关性、内容质量与覆盖度

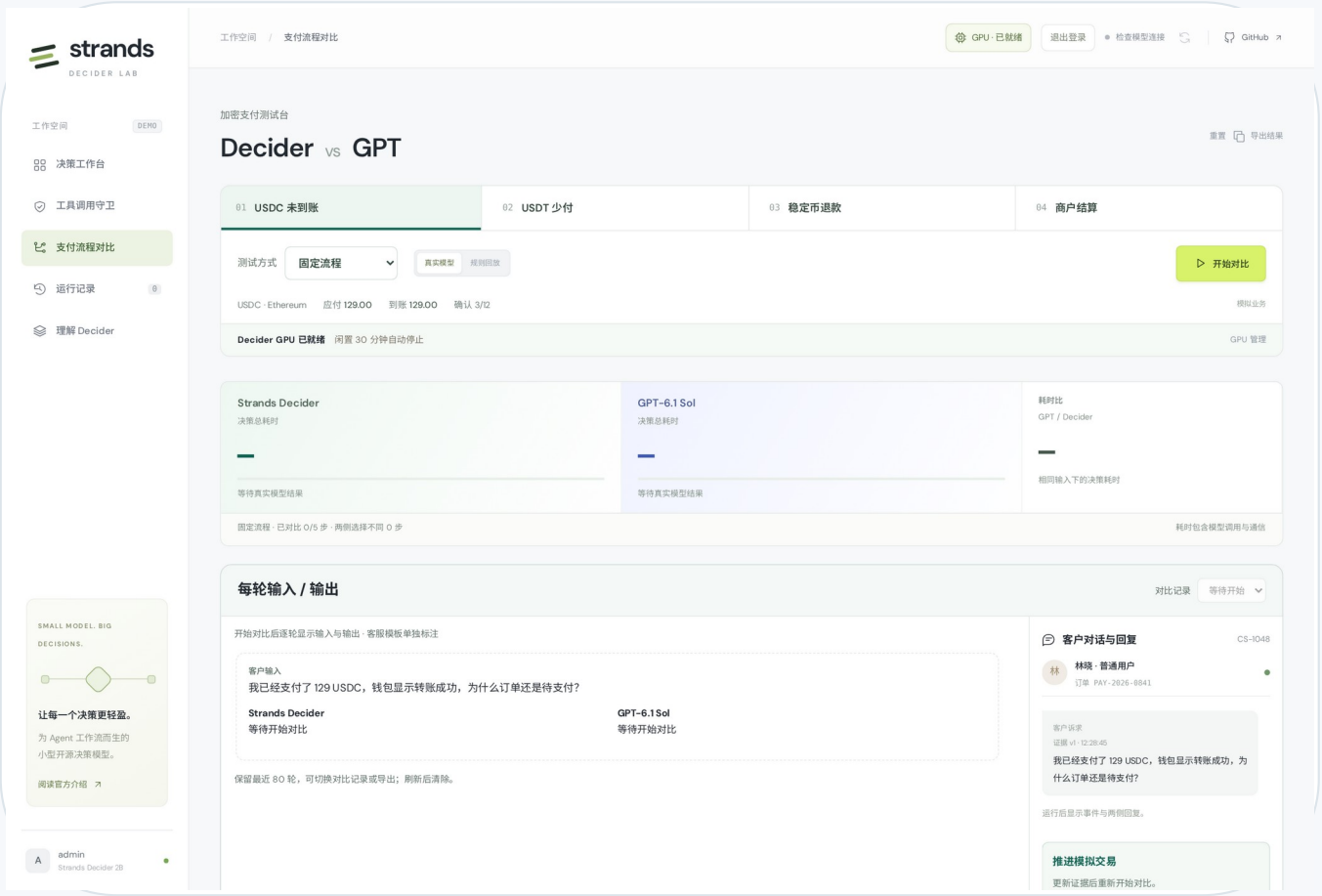
记忆与上下文管理

选择哪些片段需要保留或取回

建议组合：确定性代码校验 → Decider 常规选择 → 复杂或低置信案例交给大模型 / 人工

当前演示入口：先选测试方式，再开始对比

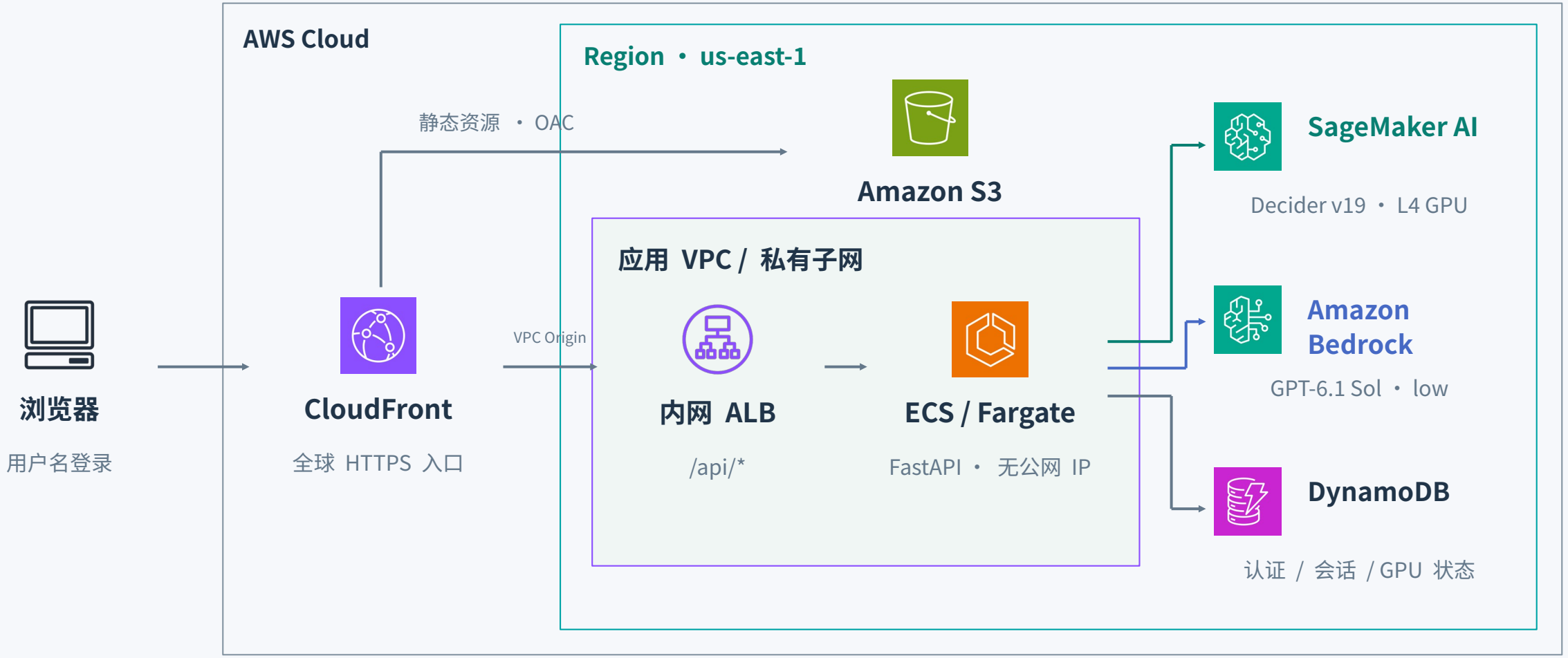
固定流程 / 自主闭环 / 仅比较决策，使用同一个「开始对比」入口。



- 顶部选择场景与测试方式
- 耗时卡片下方直接看输入 / 输出
- 客户对话与模拟事件默认可见

AWS 架构：后端直接调用三个业务服务

us-east-1 · AWS 官方服务图标 · 图中省略 Runtime / DynamoDB 私有端点节点。



S3 公共访问全阻止；后端无公网 IP 。 Bedrock 使用美国跨区域推理配置。

GPU 管理：所有模块共用一个弹窗

本次采集时 GPU 已就绪；只复用现有实例，没有发起启停。



- 真实推理前检查状态
- 唯一一组启动 / 停止按钮
- 闲置 30 分钟删除 Endpoint

四种支付场景，覆盖客服到交易处置

每个案例可修改证据、执行模拟事件，再对比两种模型在新状态下的判断。

USDC 未到账

应付 / 到账 129 USDC

链上确认 3 / 12

推进：模拟确认数达标

USDT 少付

应付 100，到账 99.80 USDT

差额 0.20

推进：模拟补足差额

稳定币退款

申请退款 250 USDC

地址未验证，尚未审批

推进：验证地址 → 审批通过

商户结算延迟

4,800 USDC，等待 120 分钟

演示 SLA 为 60 分钟

推进：模拟结算到账

模型真实调用；交易、订单、工单与账本均为沙盒模拟，不连接钱包、不广播链上交易。

两种对比方式，回答不同的问题

固定流程比较同一输入；自主闭环比较各自实际完成业务的过程。

维度	固定输入流程	自主闭环
模型输入	每步使用冻结业务证据	使用本侧最新状态 + 执行反馈
执行时机	收齐建议后按序执行	每次判断后立即执行并读取结果
外部事件	推进证据后重新开始	等待事件后恢复本次工作流
对比指标	成功且同 input_hash 的配对决策	完成状态、调用数、总耗时与重判
业务结果	处理五步也可能没有资金动作	可完成、等待、失败或转人工

闭环是一个对比测试选项；候选、执行保护与调用预算在两侧相同。

什么是“演示参考”？它是开发者写出的业务基准

代码入口： app/crypto.py 。参考答案独立计算，用于比较模型输出；不直接填入模型请求。

候选条件 • OPTIONS

开发者提供可选动作与描述
两模型读取相同事实和条件
模型负责从候选项中选择

演示参考 • reference()

代码计算本步骤的预期动作
例如：少付 → 挂起并跟进
用于统计参考一致与差异

执行保护

根据实际状态校验资金和结案
未满足条件 → 调整实际动作
保留模型的原始建议与原因

参考一致 = 原始模型 choice 与 reference_choice 相同；它衡量本演示策略符合度。

参考由开发者定义；通过风控审批检查，也仍需满足确认数与足额条件才能执行收款。

Decider 执行差异：原始选择与实际动作分别保留

本次固定流程中有 6 次 Decider 参考差异；保护调整不计为模型判断正确。

场景 / 步骤	演示参考	Decider 建议	实际采用
USDC 未到账 风控审批	规则检查通过	等待人工审批	等待人工审批 保留原建议
USDT 少付 业务处置	挂起并跟进	确认订单收款	挂起并跟进 保护调整
稳定币退款 风控审批	验证退款地址	等待人工审批	等待人工审批 保留原建议
稳定币退款 业务处置	挂起并跟进	提交模拟退款	挂起并跟进 保护调整
稳定币退款 回复结案	回复并持续跟进	回复并结案	回复并持续跟进 保护调整
商户结算 回复结案	回复并持续跟进	回复并结案	回复并持续跟进 保护调整

能观察到条件遵循偏差；仅凭 choice / 概率，不能断言某次偏差的内部根因。

为什么会不一致？模型评分不是执行规则代码

Decider 输出有限候选中的选择；符合输出结构，不等于正确遵守全部业务条件。

可核对的事实

记录模型实际 choice / 分布

记录原始输入与参考差异

执行器另行检查实际状态

不能据此推断

一次差异的内部思维过程

金融行业的通用准确率

Decider 比 BERT 更快或更准

可以改进的环节

设计清楚的候选与条件

收集失败和重判样本

按业务数据评估与校准

金额计算、地址校验和资金权限等确定性条件，仍由业务代码执行。

每轮输入 / 输出：默认展开，直接并排对照

当前页面实际截取：客户诉求、交易证据、两侧输出、实际执行与时间。

第 1 轮 证据 v1 · 固定输入 · 真实模型

输入 · 客服受理

我已经支付了 129 USDC，钱包显示转账成功，为什么订单还是待支付？

应付 129.00 · 到账 129.00　确认 3/12　无风险标记

Strands Decider	GPT-6.1 Sol
决策 91.8 ms	决策 1.17 s
模型输出	模型输出
支付客服	**支付客服**
payment_support	payment_support
实际执行	实际执行
工单 CS-1048 已受理，分类为「支付客服」；工单状态：处理中。	工单 CS-1048 已受理，分类为「支付客服」；工单状态：处理中。
12:26:25　请求往返 120.0 ms	12:26:25　请求往返 1.20 s
▶ 完整请求与返回	▶ 完整请求与返回

固定输入：同一步的选择与速度一起看

USDC 未到账：确认数 3/12；表中显示原始建议、实际动作与各步耗时。

Strands Decider	GPT-6.1 Sol	耗时比
决策总耗时	决策总耗时	GPT / Decider
458.3 ms	7.79 s	17.0×
5 次实测决策	5 次实测决策	相同输入下的决策耗时

固定流程 · 已对比 5/5 步 · 两侧选择不同 1 步

耗时包含模型调用与通信

逐步流程对比

业务步骤	Strands Decider		GPT-6.1 Sol		选择对比
01 客服受理	支付客服 实际：支付客服	91.8 ms	支付客服 实际：支付客服	1.17 s	相同
02 交易核验	等待链上确认 实际：等待链上确认	94.9 ms	等待链上确认 实际：等待链上确认	1.33 s	相同
03 风控审批	等待人工审批 实际：等待人工审批	87.4 ms	规则检查通过 实际：规则检查通过	3.77 s	不同
04 业务处置	挂起并跟进 实际：挂起并跟进	91.0 ms	挂起并跟进 实际：挂起并跟进	703.5 ms	相同
05 回复结案	回复并持续跟进 实际：回复并持续跟进	93.2 ms	回复并持续跟进 实际：回复并持续跟进	816.6 ms	相同

已处理 5/5 步，不等于业务成功

等待、挂起和继续跟进，也属于已处理步骤。



订单达到目标状态

已收款 / 已退款 / 已对账

存在对应模拟账本

金额与资金动作可核对

客服工单已结案

不能仅依据模型建议判断

退款处置：建议与真正执行的结果分开显示

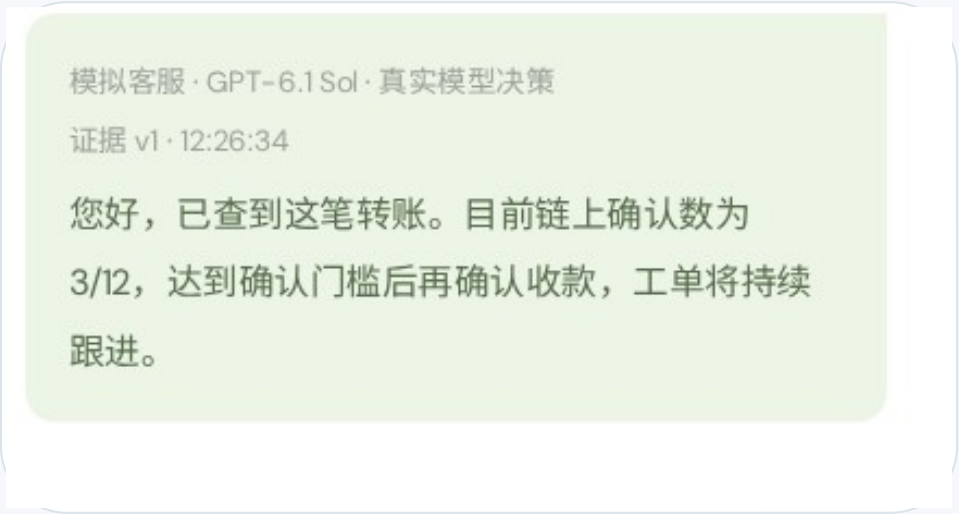
地址未验证、审批未完成时，不能因为模型选择退款就认定资金已执行。



客户对话与模拟事件，放在主区域

客服回复由模板按实际订单 / 工单状态生成，不是 GPT 长文本质量测试。

模板回复：如实说明待确认状态



未满足确认数 → 持续跟进

更新证据后重新对比

闭环等待时则继续当前运行

模拟事件与客户输入



自主闭环：判断 → 执行 → 读取结果 → 再判断

每次回到模型前，把实际状态和最近反馈写入下一轮输入。



条件未齐 → 等待

等待确认、补款、地址验证等
事件更新后再核验

无效动作 → 重判

把未执行原因传回模型
同一步两次无进展转人工

业务完成 → 结束

订单 + 账本 + 结案均确认
已记账动作不再次提交

预算：最多 20 次决策 / 32 次 tick；页面连续运行约 5 分钟后暂停。

闭环输入：带着本侧执行反馈进入下一轮

同样的初始证据，两侧执行后可能进入不同状态。



USDC 未到账闭环：如实展示完成或转人工

Decider： 业务完成 · 6 次决策； GPT： 业务完成 · 6 次决策。



Strands Decider

决策累计 0.57 s · 模拟事件 1 次

GPT-6.1 Sol

决策累计 7.27 s · 模拟事件 1 次

两侧后续输入和调用次数可能不同，此处不能解释为固定同输入的推理速度排名。

稳定币退款闭环：如实展示完成或转人工

Decider： 业务完成 · 9 次决策； GPT： 业务完成 · 9 次决策。



Strands Decider

决策累计 0.90 s · 模拟事件 2 次

GPT-6.1 Sol

决策累计 10.54 s · 模拟事件 2 次

两侧后续输入和调用次数可能不同，此处不能解释为固定同输入的推理速度排名。

闭环的停止原因，也属于演示结果

继续、等待、完成、调用失败和转人工都应可见，不能只展示成功路径。

USDC 未到账 · Decider

模拟订单、对应账本与工单结案均已确认，业务闭环完成。

USDC 未到账 · GPT

模拟订单、对应账本与工单结案均已确认，业务闭环完成。

稳定币退款 · Decider

模拟订单、对应账本与工单结案均已确认，业务闭环完成。

稳定币退款 · GPT

模拟订单、对应账本与工单结案均已确认，业务闭环完成。

前端可回看每次输入、输出、执行反馈和失败记录；继续运行追加新记录。

本次实测： 19 组成功配对，累计耗时比 12.20×

四个初始场景，每侧尝试 20 次决策；只汇总成功且输入相同的配对。

初始案例	配对	Decider 合计	GPT 合计	D 参考一致	G 参考一致
USDC 未到账	5	458.3 ms	7791.8 ms	4/5	5/5
USDT 少付	5	452.5 ms	5469.0 ms	4/5	5/5
稳定币退款	5	521.3 ms	4275.7 ms	2/5	5/5
商户结算	4	377.4 ms	4531.6 ms	3/4	4/4
合计	19	1809.6 ms	22068.2 ms	13/19	19/19

Bedrock 有 1 次读取超时，单独保留错误记录；未使用回放补结果。

普通大模型也可能失败：结果与可用性都要记录

本次商户结算 / 业务处置： Bedrock ReadTimeoutError 。

观察到的事实

Bedrock 返回 4/5 个决策
业务处置请求读取超时
该侧没有执行完整沙盒方案
错误与本轮输入保留

比较时如何处理

速度仅统计成功同输入配对
失败数量单独披露
不能把成功请求延迟当总体体验
不能用规则结果替代失败输出

这是单次实际故障记录，不足以推断任何模型的长期可用性。

计时与成本：先说清口径，再判断收益

本项目已经展示多种计时；它们回答的问题不同。

每步 / 累计后端耗时

观察模型调用开销
包括通信、处理和排队
五步分别求和
用于同输入配对比较

浏览器经过时间

观察实际交互等待
同一步的两模型并发
整轮还含沙盒执行
与累计模型耗时分开显示

计费与运行成本

SageMaker： GPU 运行时间
Bedrock： 模型用量
另计 API 、网络与存储
需要结合利用率评估

本轮没有测量 P95 / P99 、并发吞吐、长期稳定性或每次决策的美元成本。

原始输入输出可追溯，生产状态还需持久化

当前演示已实现受限闭环；金融生产系统还需要服务端工作流与幂等。

当前已实现	生产化还需补齐
判断 → 执行 → 反馈 → 重判 / 等待	服务端持久化状态、可靠队列与恢复调度
当前运行内避免重复资金步骤	数据库级唯一约束、状态版本与条件写
逐轮输入输出、错误历史、导出	持久审计、权限隔离与敏感数据治理
明确标记的模拟确认 / 补款 / 审批	可验证的链上事件、支付账务和授权审批
预算限制、连续无进展转人工	持续评测、费用预算与人工接管流程

自动按置信度切换到 GPT 属于后续方案；当前不会静默替换 Decider 输出。

现场演示：先看同输入，再看执行反馈

演示入口：支付流程对比 → 测试方式 → 开始对比。

00:00

检查 GPU

顶部公共入口；未就绪会弹窗，等就绪后返回运行。

00:45

固定流程对比

看每轮输入与两模型输出，再看实际动作和业务是否完成。

02:00

切换自主闭环

开启自动模拟证据；逐轮观察反馈、事件、重判与耗时。

03:30

解释分歧与停止

完成、等待或转人工均保留；不要只用 5/5 判断成功。

04:30

回看历史并导出

切换对比记录，展开原始请求与返回，保存 JSON 。

https://d3i7ogfx5ml7dc.cloudfront.net/#crypto

结论：快的决策，需要可观察、可约束的执行

Decider 适合高频有限候选判断；能否完成业务，要看实际状态和反馈。

模型层

候选与条件来自应用

Decider 直接评分， GPT 生成 JSON

BERT 固定分类头有不同适配方式

执行层

原始选择与实际动作分开

业务完成用订单、账本、结案判定

反馈闭环可改进，也可能转人工

评测层

同输入统计与闭环统计分开

成功延迟与调用失败同时披露

用真实业务分布继续验证

推荐组合：代码做确定性校验，模型做语义选择，高影响操作由明确权限与人工流程控制。

资料来源、截图与本次数据

原版文件不覆盖；所有新版截图来自当前生产界面，数据来源与采集时间保留。

Strands 官方博客 / v19 模型卡

模型目的、 Pointer Head 、问题类型与限制

https://strandsagents.com/blog/introducing-strands-decider/

Decider 锁定源码

判断计算、 confidence 与实际请求协议

https://github.com/strands-labs/strands-decider/tree/63d24ae286e50105fba0bd1db15b0e243aea4650/src/strands_decider

BERT 论文与分类器

BERT 对比未进行性能实测

https://aclanthology.org/N19-1423/

项目与在线演示

代码、架构、闭环实现及用户操作

https://github.com/milan9527/StrandsDecider

本次证据

benchmark-2026-10-10.json / loop-observations.json / capture-manifest.json

https://github.com/milan9527/StrandsDecider/tree/main/docs/presentations/v2